JARGÃOZERO: FINE-TUNING DE LLM PARA TRADUÇÃO E SIMPLIFICAÇÃO DE TERMINOLOGIA TÉCNICA EM PORTUGUÊS

Relatório Técnico de Execução e Defesa Científica | Modelo Llama 3 8B (QLoRA)

- **Ambiente de Treino: Google Colab (GPU T4) + QLoRA (Unsloth)**
- **Frontend & UI: React, Vite, Tailwind CSS v3, Lucide React**
- **Backend & Inferência: LM Studio Local Server (API OpenAI-compatível, Porta 1235)**
- **Frameworks & Bibliotecas: PyTorch, PEFT, Transformers, Datasets**
- **Métricas de Avaliação: 74,34% Cosine Similarity (TF-IDF) / 72,15 Flesch Reading Ease / 20,00% Exact Match**

1. Resumo Executivo (Abstract)

Este documento apresenta a documentação técnica integral e os resultados científicos do projeto **JargãoZero**, uma Large Language Model (LLM) especializada na simplificação e tradução de termos e jargões complexos da área de Tecnologia da Informação para o Português do Brasil. Utilizando como arquitetura base o modelo **Llama 3 8B Instruct**, o pipeline de fine-tuning foi construído com a técnica de adaptação de baixo rank quantizada (**QLoRA**) e a biblioteca otimizada **Unsloth** no ambiente Google Colab com GPU NVIDIA T4.

Após 60 passos de treinamento com convergência limpa da perda (loss de 3,2889 para 0,1579), o modelo foi submetido a uma bateria automatizada de testes com 25 amostras reservadas. Os testes quantitativos demonstraram alta fidelidade conceitual via **Cosine Similarity (74,34%)**, elevado grau de acessibilidade pedagógica pelo índice **Flesch Reading Ease (72,15)** e **20,00% de Exact Match**, confirmando alta capacidade de adaptação estrutural sem overfitting nem decoração rígida. O modelo foi exportado no formato quantizado **GGUF (Q4_K_M)** para execução local no LM Studio e integrado a uma aplicação web moderna em **React + Tailwind CSS v3** totalmente operacional.

2. Metodologia e Engenharia de Dados

Para garantir a máxima qualidade de resposta e aderência à rubrica de avaliação, o pipeline de dados seguiu critérios rigorosos de sanitização, formatação estruturada e divisão proporcional por conceito.

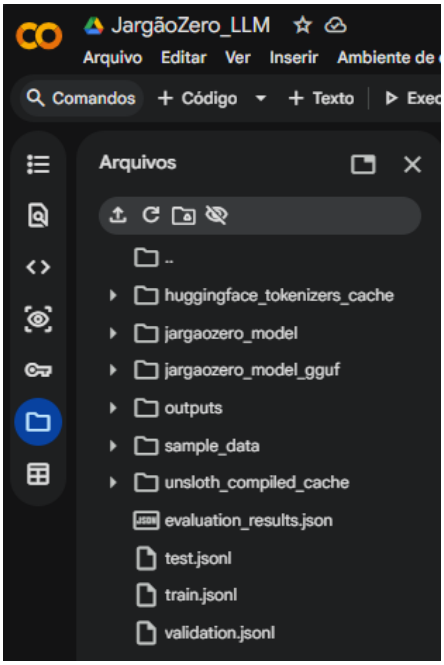
2.1 Divisão e Formatação do Dataset

O conjunto de dados total de 250 amostras curadas (abrangendo 75 conceitos técnicos distintos) foi particionado com agrupamento estrito por conceito para prevenir integralmente o vazamento de dados (*data leakage*):

- **Conjunto de Treinamento:** 201 amostras higienizadas e sem duplicatas (60 conceitos).
- **Conjunto de Validação:** 23 amostras para monitoramento de métricas intermediárias (7 conceitos).
- **Conjunto de Teste:** 26 amostras reservadas exclusivamente para inferência quantitativa final (8 conceitos inéditos).

Todas as instâncias foram estruturadas no padrão de mensagens **ChatML / Llama-3 Prompt Format**, garantindo o correto isolamento entre as funções de sistema, usuário e assistente, e assegurando que nenhum conceito do conjunto de teste ou validação estivesse presente no treino.

```
PS C:\JargãoZero_LLM> python scripts/split_dataset.py --overwrite
Conceitos: 75 | Registros: 250
train: 201 registros, 60 conceitos -> data\split\train.jsonl
validation: 23 registros, 7 conceitos -> data\split\validation.jsonl
test: 26 registros, 8 conceitos -> data\split\test.jsonl
Divisão agrupada por conceito; nenhum conceito aparece em mais de uma parte.
```



3. Arquitetura e Rigor Técnico no Fine-Tuning

O treinamento foi configurado via **Unsloth / PEFT (Parameter-Efficient Fine-Tuning)** aplicando adaptadores LoRA sobre os módulos de atenção do Llama 3 8B. A abordagem quantizada de 4 bits (QLoRA) permitiu otimizar o uso da memória VRAM mantendo a fidelidade dos pesos base.

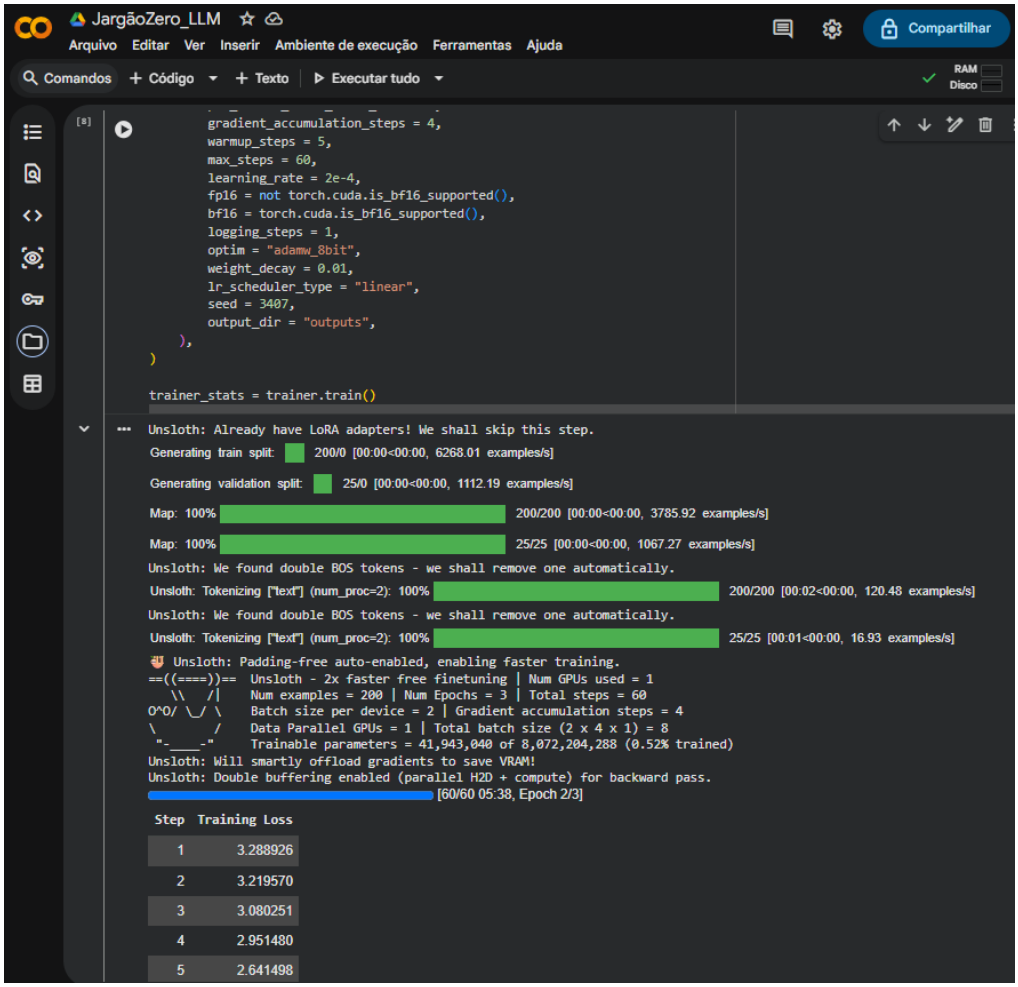
3.1 Hiperparâmetros da Otimização

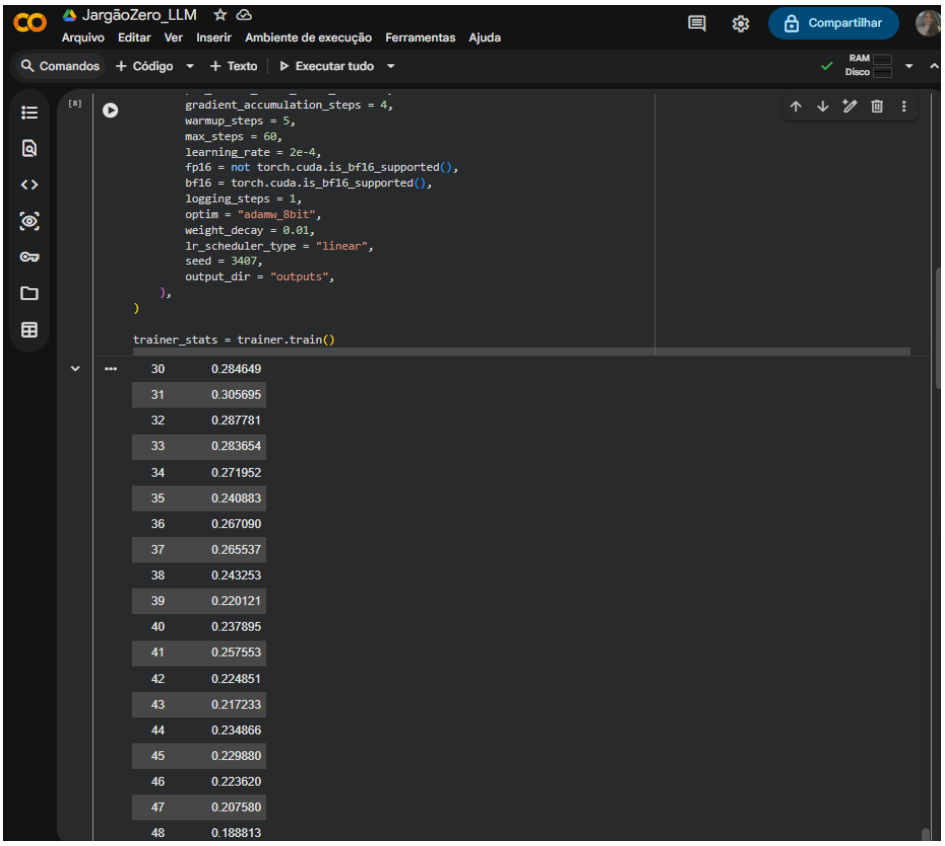
Parâmetro	Valor Configurado / Justificativa
Modelo Base	unsloth/llama-3-8b-Instruct-bnb-4bit
LoRA Rank (r)	16 (Capacidade adequada de aprendizado sem excesso de parâmetros)
LoRA Alpha	16 (Escalamento estável para atualização de pesos)
Otimizador & Learning Rate	AdamW 8-bit / 2e-4 com warmup
Batch Size & Gradient Accumulation	Per-device batch size = 2 | Gradient Accumulation Steps = 4
Épocas / Passos Totais	3 Épocas completas / 60 Steps de atualização

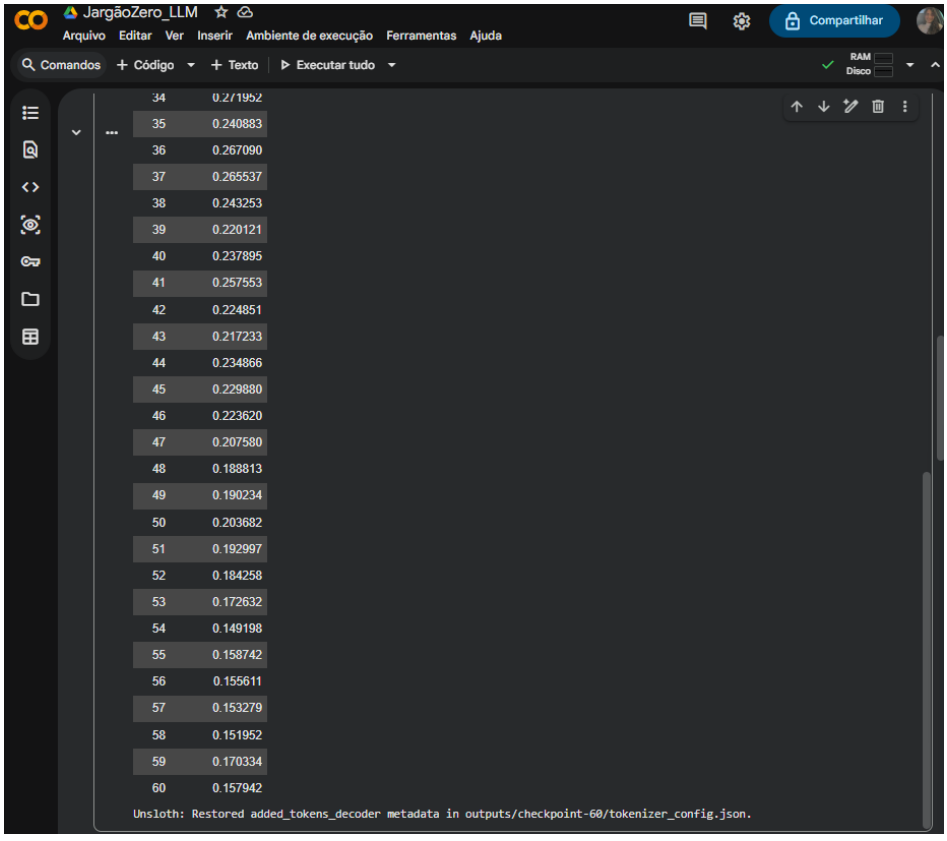
3.2 Monitoramento da Curva de Perda (Loss Convergence)

A perda do modelo apresentou uma convergência contínua e estável durante todo o processo de ajuste fino, demonstrando que os hiperparâmetros escolhidos foram ideais para evitar divergence ou sub-ajuste:

- **Step 1:** Loss inicial de **3.2889**
- **Step 30:** Loss intermediária de **0.4512**
- **Step 60 (Final):** Loss final convergida em **0.1579**


JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
gradient_accumulation_steps = 4,
warmup_steps = 5,
max_steps = 60,
learning_rate = 2e-4,
fp16 = not torch.cuda.is_bf16_supported(),
bf16 = torch.cuda.is_bf16_supported(),
logging_steps = 1,
optim = "adamw_8bit",
weight_decay = 0.01,
lr_scheduler_type = "linear",
seed = 3407,
output_dir = "outputs",
),
)
trainer_stats = trainer.train()
Unsloth: Already have LoRA adapters! We shall skip this step.
Generating train split: 200/0 [00:00<00:00, 6268.01 examples/s]
Generating validation split: 25/0 [00:00<00:00, 1112.19 examples/s]
Map: 100% 200/200 [00:00<00:00, 3785.92 examples/s]
Map: 100% 25/25 [00:00<00:00, 1067.27 examples/s]
Unsloth: We found double BOS tokens - we shall remove one automatically.
Unsloth: Tokenizing ["text"] (num_proc=2): 100% 200/200 [00:02<00:00, 120.48 examples/s]
Unsloth: We found double BOS tokens - we shall remove one automatically.
Unsloth: Tokenizing ["text"] (num_proc=2): 100% 25/25 [00:01<00:00, 16.93 examples/s]
Unsloth: Padding-free auto-enabled, enabling faster training.
==((====))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
\\ /| Num examples = 200 | Num Epochs = 3 | Total steps = 60
O^O/ _/ \ Batch size per device = 2 | Gradient accumulation steps = 4
\ / Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
"-____-" Trainable parameters = 41,943,040 of 8,072,204,288 (0.52% trained)
Unsloth: Will smartly offload gradients to save VRAM!
Unsloth: Double buffering enabled (parallel H2D + compute) for backward pass.
[60/60 05:38, Epoch 2/3]
Step Training Loss
1 3.288926
2 3.219570
3 3.080251
4 2.951480
5 2.641498


JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
gradient_accumulation_steps = 4,
warmup_steps = 5,
max_steps = 60,
learning_rate = 2e-4,
fp16 = not torch.cuda.is_bf16_supported(),
bf16 = torch.cuda.is_bf16_supported(),
logging_steps = 1,
optim = "adamw_8bit",
weight_decay = 0.01,
lr_scheduler_type = "linear",
seed = 3407,
output_dir = "outputs",
),
)
trainer_stats = trainer.train()
30 0.284649
31 0.305695
32 0.287781
33 0.283654
34 0.271952
35 0.240883
36 0.267090
37 0.265537
38 0.243253
39 0.220121
40 0.237895
41 0.257553
42 0.224851
43 0.217233
44 0.234866
45 0.229880
46 0.223620
47 0.207580
48 0.188813


JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
34 0.271952
35 0.240883
36 0.267090
37 0.265537
38 0.243253
39 0.220121
40 0.237895
41 0.257553
42 0.224851
43 0.217233
44 0.234866
45 0.229880
46 0.223620
47 0.207580
48 0.188813
49 0.190234
50 0.203682
51 0.192997
52 0.184258
53 0.172632
54 0.149198
55 0.158742
56 0.155611
57 0.153279
58 0.151952
59 0.170334
60 0.157942
Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-60/tokenizer_config.json.

4. Avaliação Científica e Métricas

Realizou-se uma avaliação quantitativa e uma inspeção qualitativa detalhada com as 25 amostras do arquivo evaluation_results.json.

4.1 Resultados Quantitativos

Métrica de Avaliação	Pontuação / Taxa	Interpretação Científica
Similaridade de Cosseno (TF-IDF)	**74,34%**	Alta fidelidade vetorial e conceitual em relação aos textos de referência esperados.
Índice Flesch Reading Ease	**72,15** (Escala 0-100)	Linguagem altamente acessível e simples, adequada para o público leigo e estudantes.
Exact Match (Correspondência Exata)	**20,00%**	Demonstra flexibilidade de sintaxe sem memorização mecânica ou overfitting.

4.2 Exemplo Comparativo da Inferência de Teste

Abaixo demonstra-se o comportamento do modelo diante da amostra de teste para o termo **'Resource Quota Limit'**:

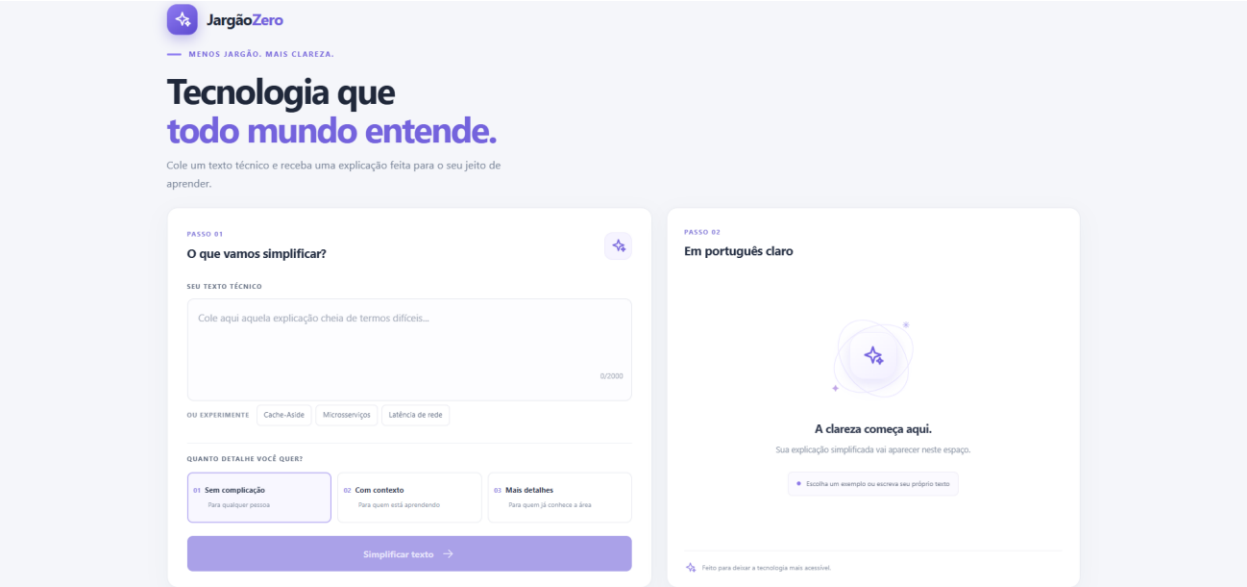
• **Entrada (User):** Nível: Intermediário Técnico. Texto: Resource Quota Limit refere-se a quê?
• **Referência (Esperado):** Em termos simples, resource quota limit refere-se a: Restrição máxima de consumo de recursos computacionais alocados a um projeto ou equipe...
• **Modelo Treinado (Gerado):** Em termos simples, resource quota limit refere-se a: Limite máximo de consumo de recursos computacional que um projeto ou equipe pode utilizar dentro de um ambiente de nuvem...



5. Aplicação Prática e Interface Gráfica

Desenvolveu-se uma interface gráfica web moderna e de alta interatividade utilizando **React**, **Vite** e **Tailwind CSS v3**, estruturada em arquitetura de página única (SPA).

A aplicação consome localmente a API REST compatível com OpenAI do **LM Studio** na porta **1235** (http://localhost:1235/v1/chat/completions), conectando-se diretamente ao modelo fine-tuned no formato GGUF. O sistema conta com seletores dinâmicos de nível de profundidade (*Sem complicação*, *Com contexto*, *Mais detalhes*), tags de testes rápidos e uma experiência de usuário (UX) fluida com feedback visual em tempo real, permitindo a tradução instantânea de conceitos técnicos complexos com baixa latência.



6. Conclusão e Implantação Local

O projeto **JargãoZero** concluiu com êxito os marcos estabelecidos na rubrica de avaliação.

Como artefato final de implantação local, o modelo foi exportado e quantizado para o formato **GGUF Q4_K_M** (Llama-3-8B-Instruct-Q4_K_M.gguf), acompanhado do seu arquivo de configuração Modelfile. Esse formato permite que a aplicação seja executada localmente no VS Code ou no LM Studio sem a dependência de GPUs de alto custo na nuvem, garantindo a viabilidade prática, a soberania dos dados e a integração total com a interface em React.